

Backend Intern Take-Home: Build a Minimal LLM Gateway

Time: One weekend. Budget ~10–14 focused hours. A small thing that fully works and is well-reasoned beats a big thing that half-works. Cutting scope is expected — tell us what you cut and why.

What this is really testing: Can you stand up a real backend service that sits in front of LLMs, reason about the moving parts from first principles, research what you don't already know, and get it to prod — using AI as a force multiplier without losing control of what you built. This is a deliberately small version of work you'd actually do here.

The goal

Build a small HTTP service — an **LLM gateway** — that sits between callers and one or more LLM providers and adds the things a team needs to run LLMs safely in production: **virtual keys, per-key budgets, usage/spend logging, and graceful fallback**. Then **deploy** it to a real URL.

Think of it as a stripped-down version of a production LLM gateway. You are not rebuilding a full gateway in a weekend — you're building the smallest thing that genuinely does the job, and reasoning clearly about the rest.

Core requirements (must-haves)

1. **Proxy an LLM call.** Expose an endpoint that forwards a chat/

- completion request to a real provider and returns the response. Support streaming **or** non-streaming — pick one, and tell us why.
2. **Virtual API keys.** Callers authenticate with a gateway-issued key, never the raw provider key. **Real provider credentials live server-side only** and must never reach the client or the repo.
 3. **Per-key budget enforcement.** Each key has a cap (requests, tokens, or ₹ cost — your choice). Over budget → the gateway rejects cleanly (e.g., 429/402). It must **actually block**, not silently pass through.
 4. **Usage & spend logging.** Persist per-request usage — key, model, tokens in/out, estimated cost, timestamp — to a datastore. You must be able to answer *"how much has key X spent?"*
 5. **Fallback / resilience.** When the primary provider errors or times out, do something sensible: retry, fall back to a second provider or a local/mock model, or fail fast with a clear error. Your policy — defend it.
 6. **Deploy to a real URL** with config and secrets handled properly (nothing sensitive in the repo).

Explicitly OPEN (this is where we read your judgment)

- Language, framework, datastore, host.
- Requests vs. tokens vs. ₹ for budgeting, and how you estimate cost.
- Sync vs. async logging, and what you trade off.
- How much of a real provider's request/response schema you mimic vs. your own.
- Whether you cache, and how.

Explicitly NOT required (do not gold-plate)

- **A polished frontend. We are not scoring UI.** A single admin endpoint (e.g., `GET /usage?key=...`) or a barebones page to view spend is plenty. Spend zero weekend hours on design.

- Multi-tenant auth beyond simple keys.
- Actually finetuning a model (see stretch).

Stretch (optional — attempt at most one, and only if the core fully works)

Signal, not a requirement. A rock-solid core with no stretch beats a broken everything.

- **Route to a local model** (Ollama / vLLM) as one of the providers, and send classification-type traffic to it. Watch the gotchas (e.g., context-window defaults).
 - **Add a router** that picks cheap-vs-capable per request, and justify the policy.
 - **Add semantic caching** and measure the hit rate / cost saved.
-

What to submit

1. **The deployed gateway URL**, plus a working example request (curl is fine).
2. **The source code** (public or shared repo).
3. `DECISIONS.md` — the most important deliverable. Spec below.
4. `AI-LOG.md` — how you used AI to build it. Spec below.
5. *(Optional but recommended)* a **≤ 5 min screen recording** walking us through the request lifecycle and your key decisions.

`DECISIONS.md` must cover

- **What you built**, in 3–4 sentences.
- **Moving parts & request lifecycle**. Trace one request end to end: client → gateway → auth/budget check → provider → usage logging → response. A rough diagram is welcome. *This is the system-design signal — make it clear.*

- **Your 3–5 most important decisions.** For each: the options considered, what you picked, and the tradeoff you knowingly accepted.
- **First-principles question:** *why enforce budgets at the gateway instead of trusting callers?* Answer it in your own words.
- **Concurrency:** what happens if two requests on the same near-exhausted key hit at once? Did you handle it, or knowingly not?
- **Fallback policy** and why.
- **What you deliberately did NOT build**, and why.
- **The one decision you're least confident about** — argue both sides.
- **Where it breaks**, and what you'd do with one more week.

AI-LOG.md must cover

- Which AI tools/models you used, and for what.
 - One place the AI was **wrong or misleading**, and how you caught it.
 - One place you **overrode** the AI's suggestion, and why.
 - How you stayed **in control** of code you didn't type by hand — especially anything touching secrets, budget logic, or the provider call.
 - Something you **had to learn from scratch** this weekend, and how you got up to speed.
-

Ground rules on AI

- **Using AI, LLMs, and coding agents is expected and encouraged.** This is a test of whether you can *direct* AI to build a real backend service and stay in command — not whether you can code unaided.
 - **Be honest in AI-LOG.md.** "I generated most of it" is fine; we care that you can explain and defend it.
 - **Don't ship anything you can't explain.** If you can't say why a piece exists, cut it.
-

What we're evaluating

Backend correctness and system design; first-principles reasoning; prod-readiness and resilience; and how well you wield AI, research, and learn on the go. **We do not score UI.** A rough service with a sharp `DECISIONS.md` beats a slick one you can't defend.

FAQ

- **Can I start from a boilerplate?** Yes — say so in `DECISIONS.md`.
- **Which provider?** Any. Free tiers or your own key. If you can't get a key, a local model (Ollama) as the "provider" is fully acceptable.
- **What if I don't finish?** Ship what works + `DECISIONS.md` on where you stopped and why. A well-reasoned 70% beats a mysterious 100%.
- **Do I need tests?** Only if you think they matter here. If you skip them, name it as a deliberate cut.

Submitting

Reply to this email with your deployed URL + repo within 2 days of getting the assignment. Be ready to walk us through any decision on a follow-up call. Questions - feel free to email me at nj@eazyapp.tec